

following:

- *if*
- *if-else*
- *nested if*
- *if-elif-else*

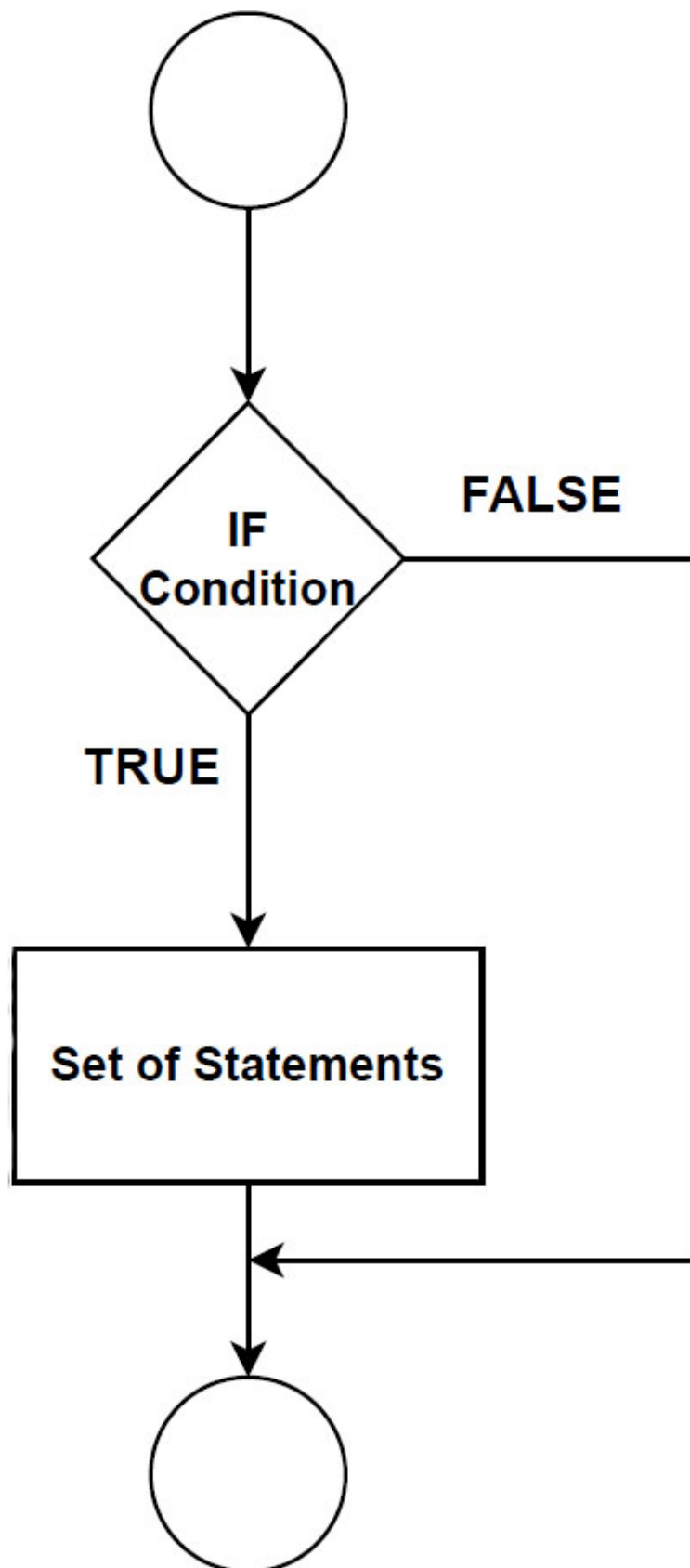
If Statements

If statements are utilized in programming to determine whether or not a particular section of code should be executed. If a condition is met, the action will be successful; otherwise, it will not.

Syntax:

```
if condition:  
    # Statements to execute if  
    # Condition is true
```

The flow diagram for the if statement is shown in Fig. [\(1\)](#).

**Fig. (1))**

Flow diagram of if statement.

Example:

```
1. # Program to check whether a number is positive or Negative
2. n = 10
3. if n > 0:
4.     print (n, "is a positive number.")
5. n1 = -2
6. if n1 < 0:
7.     print (n1, "is a Negative number.")
```

Output:

```
10 is a positive number.
-2 is a Negative number.
```

If-else Statements

The **if-else** statement is used to execute a specific code based on the results of previous statements known as if and else. If a condition is met, the code included within an if block will be carried out. If a condition is not met, the code contained within an else block will be carried out.

Syntax:

```
if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
    # condition is false
```

The flow diagram for the **if-else** expression may be found in Fig. (2).

Example:

```
1. # Program to check whether the number is positive or negative
2. n = 10
3. if n >= 0:
4.     print ("Positive or Zero")
5. else:
6.     print ("Negative number")
```

Output:

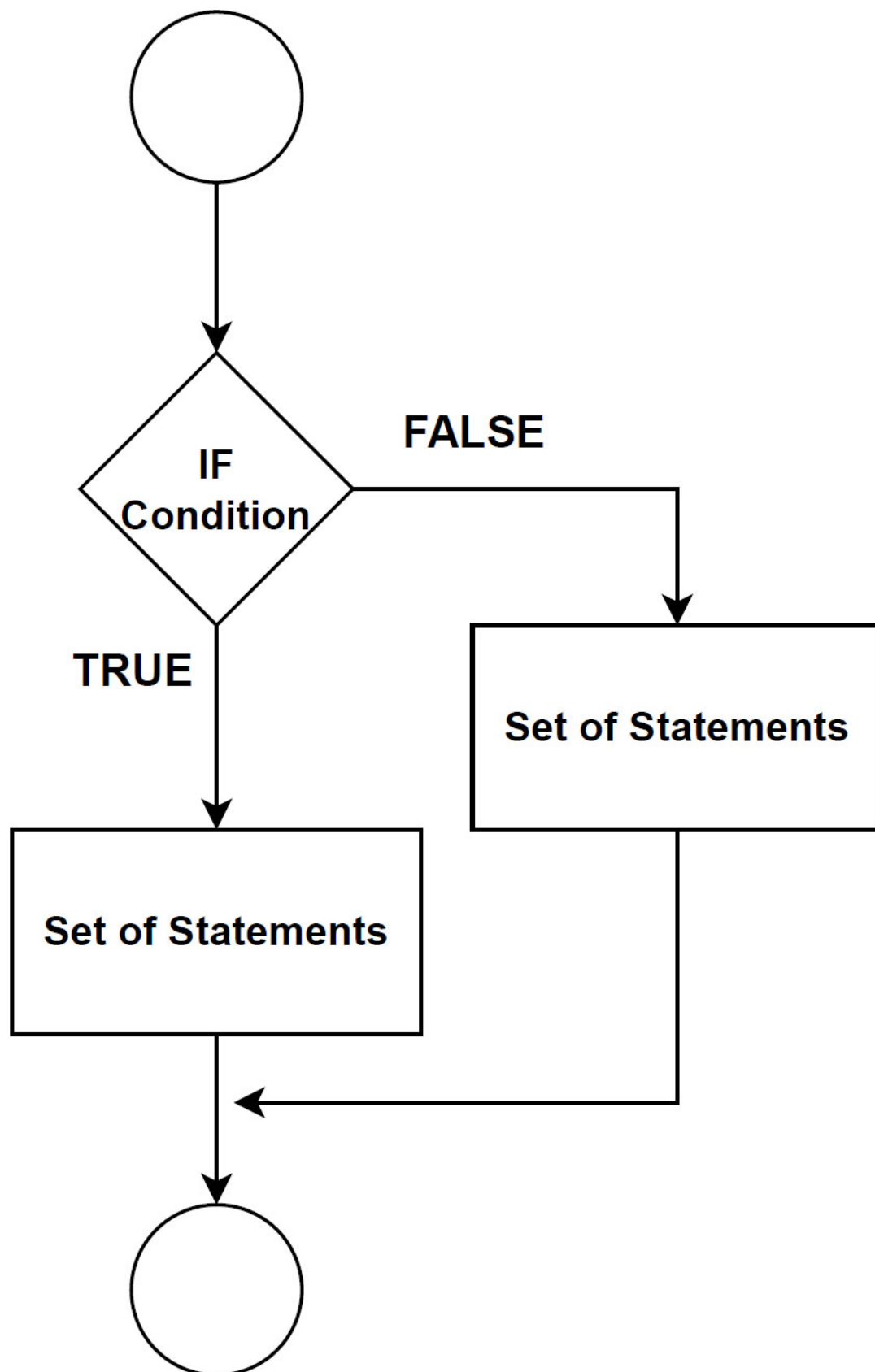
Positive or Zero
less than 1000

Nested if

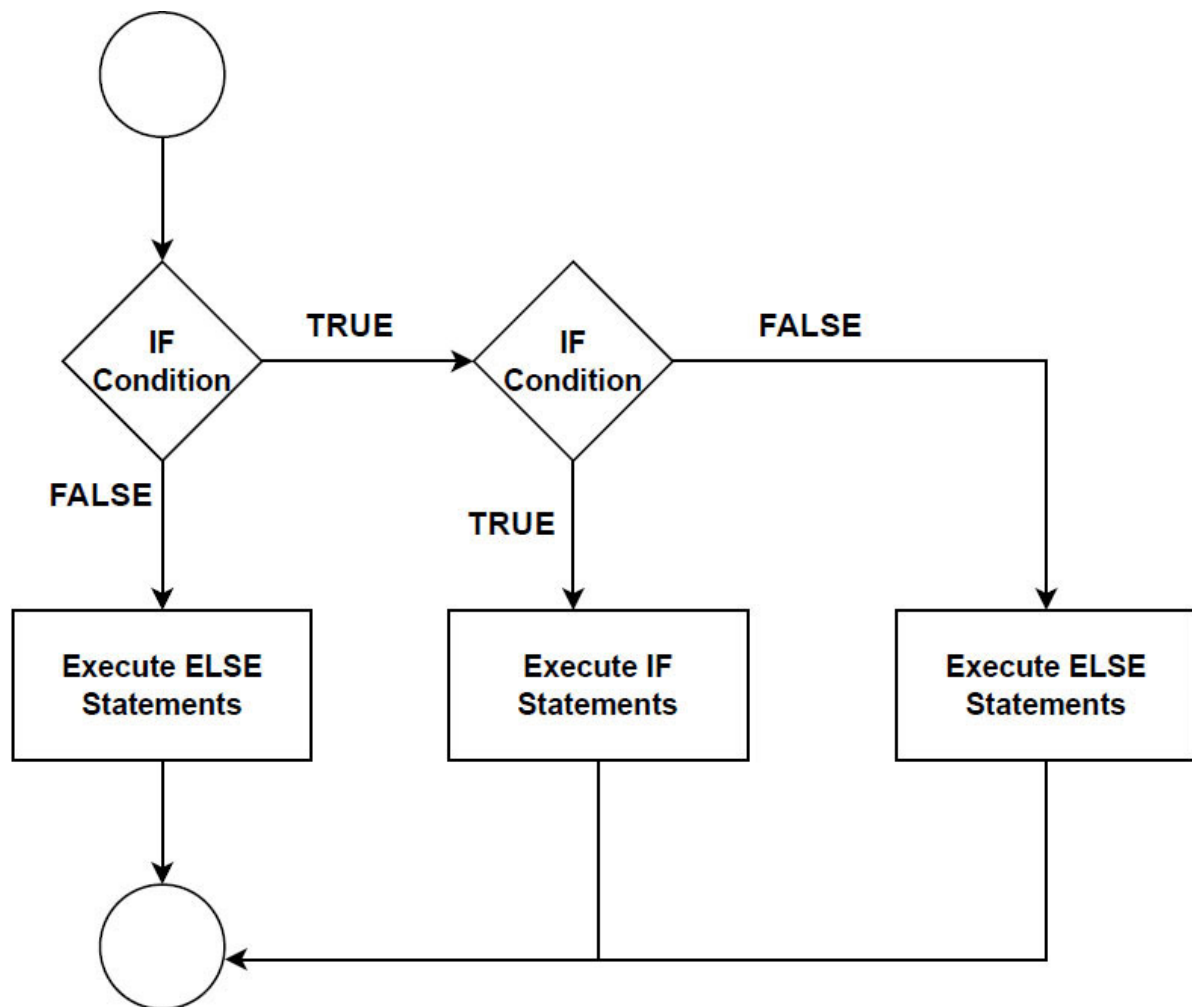
When you have an if statement inside of another if statement, you have a nested if statement. The flow pattern of nested if statements may be seen in Fig. [\(3\)](#).

Syntax:

```
if (condition1):  
    # Executes when condition1 is true  
    if (condition2):  
        # Executes when condition2 is true  
    # if block is end here  
# if block is end here
```

**Fig. (2))**

Flow diagram of *if-else* statement.

**Fig. (3))**

Flow diagram of nested if statement.

Example:

```

1. n = 10
2. if n >= 0:
3.     if n == 0:
4.         print("Zero")
5.     else:
6.         print ("Positive number")
7. else:
8.     print ("Negative number")
  
```

Output:

Positive number

if-elif-else

The if-elif-else statement is used to execute a statement or a block of statements conditionally. Fig. (4) shows the flow diagram of the if-elif-else statement.

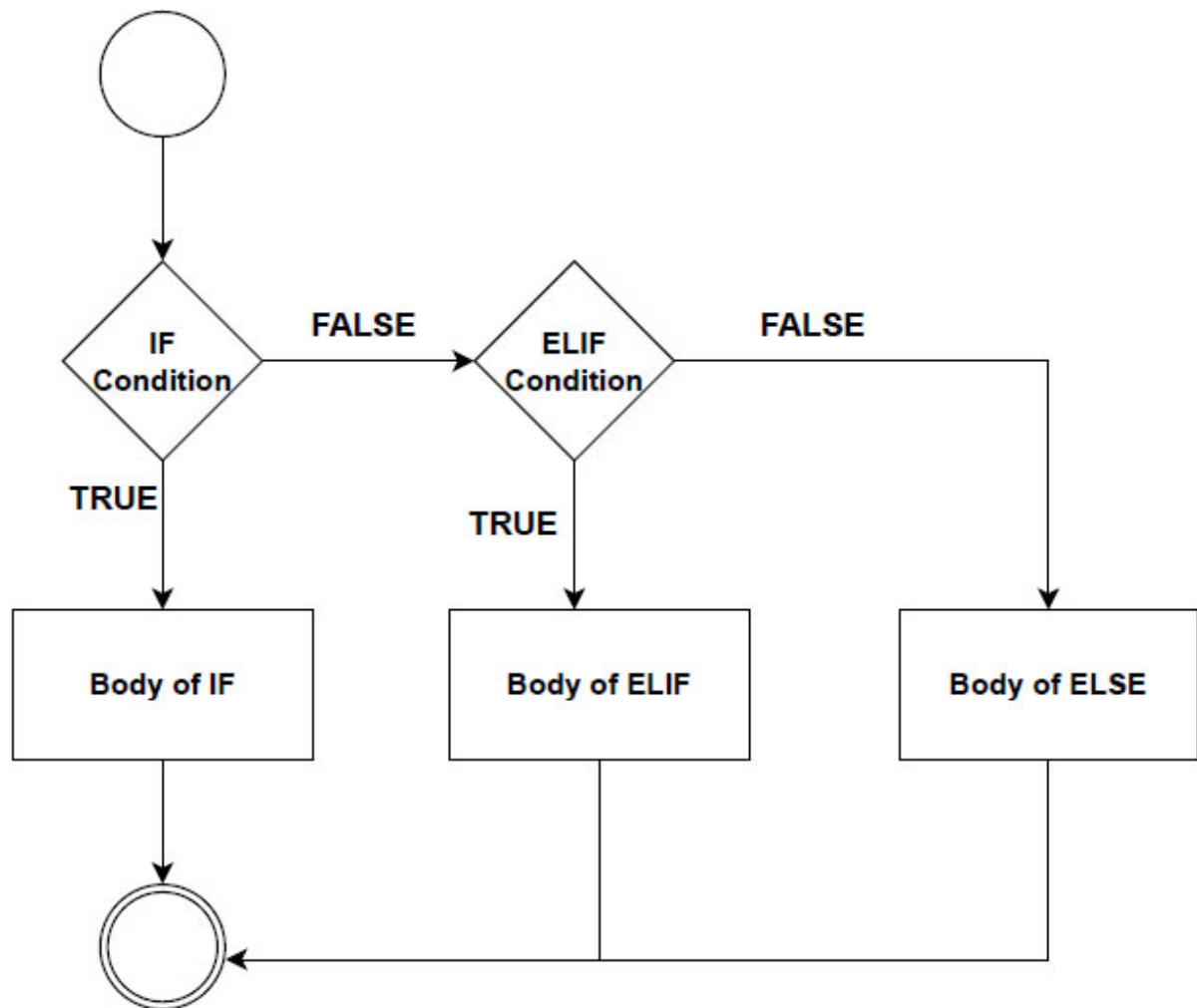


Fig. (4))

Flow diagram of if-elif-else statement.

Syntax:

```

if (condition):
    statement
elif (condition):
    statement
.
.
else:
    statement
  
```

Example:

```
1. n = 200
2. if n > 0:
3.     print ("Positive number")
4. elif n == 0:
5.     print ("Zero")
6. else:
7.     print ("Negative number")
```

Output:

```
Positive number
```

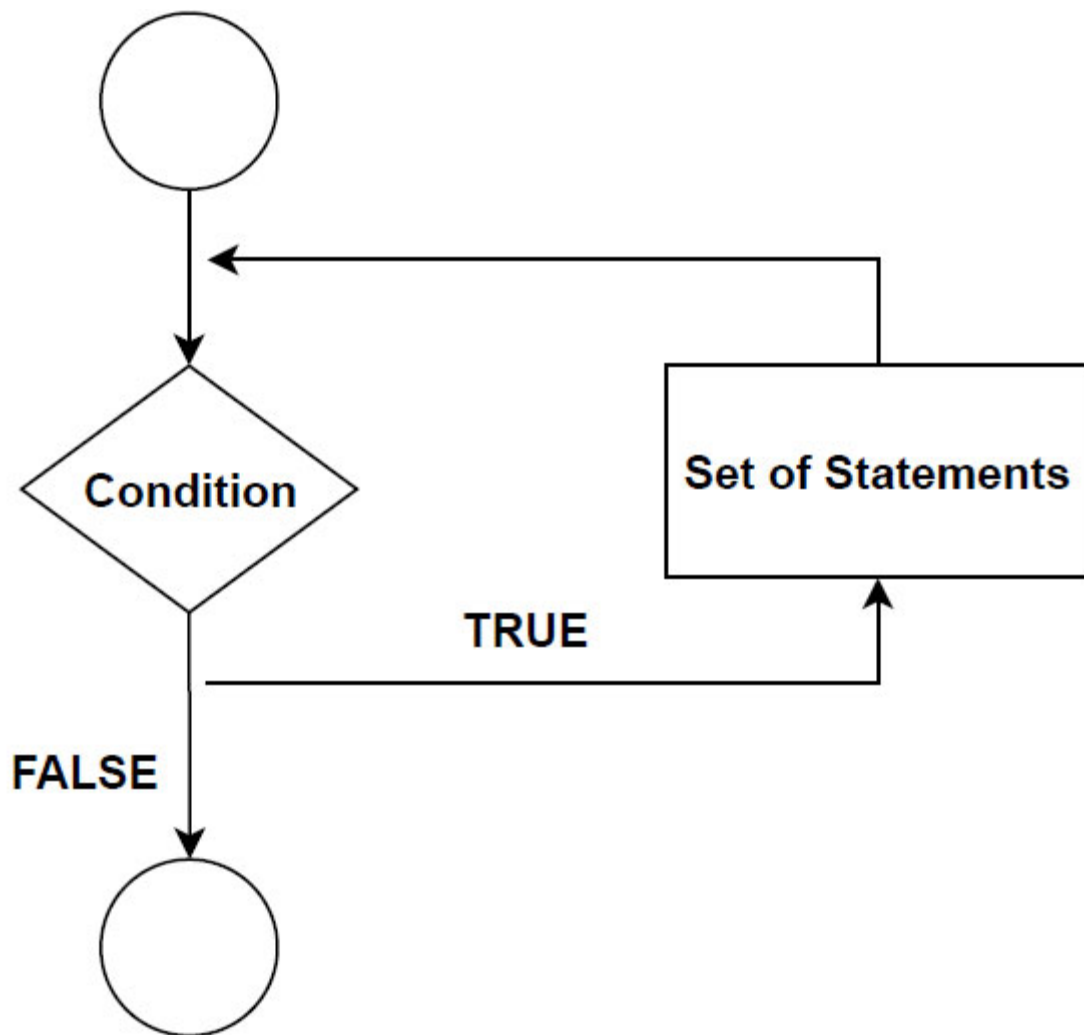
LOOPING STATEMENTS

Looping, which is just repeating the same code section many times, requires its utilization. This control structure will be utilized whenever there is a need to repeat a set of instructions based on the presence of a particular condition. Python supports two distinct iteration styles for its loops.

- *for loop*
- *while loop*

For Loop

Repeating over a sequence that is either a list, tuple, dictionary, or set may be accomplished with the help of a for loop. It is very important to remember that the ***in*** keyword is a component of the syntax for the for statement, even though it has no practical connection to the in operator used for membership checking. The flow diagram of the loop is shown in Fig. (5).

**Fig. (5))**

Flow diagram of for loop.

Syntax:

```
for var in sequence:
    statements(s)
```

Example 1:

```
1. # Python program to repeat over range 0 to n-1
2.   n = 10
3.   for i in range(0, n):
4.       print (i)
```

Output:

```
0
1
2
3
4
5
6
7
8
9
```

Example 2:

```
1. list = [1,2,3,4,5,6,7,8,9,10]
2. n = 5
3. for i in list:
4.     c = n*i
5.     print (c)
```

Output:

```
5
10
15
20
25
30
35
40
45
50
```

Example 3:

```
1. for x in range(5):
2.     print(x)
3.
4. for x in range(3, 6):
5.     print(x)
6.
7. for x in range(3, 8, 2):
8.     print(x)
```

Output:

0
1
2
3
4
3
4
5
3
5
7

While Loop

It repeats the statement until a given condition is satisfied. If the condition is true, the while loop will work; otherwise, it will not. The flow diagram of the while loop is shown in Fig. (6).

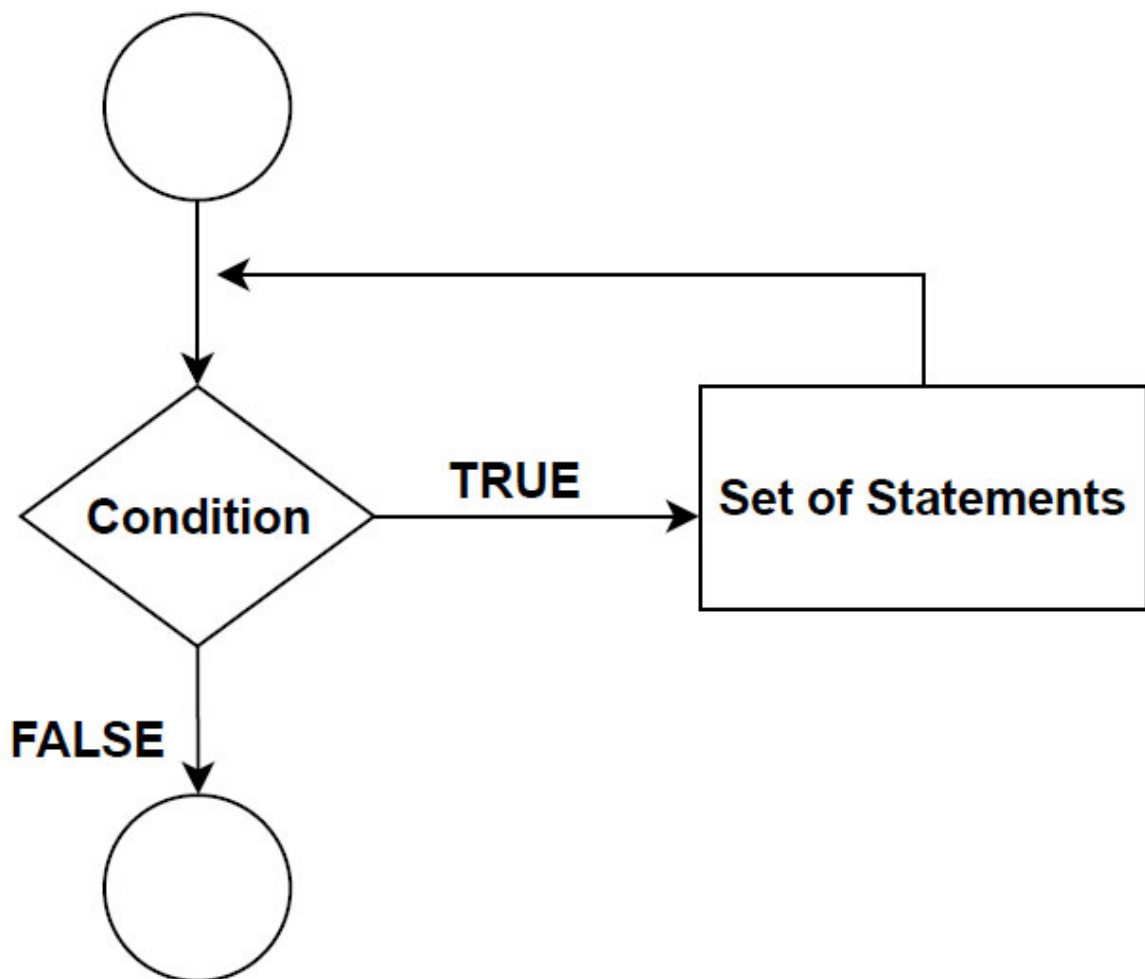


Fig. (6))

Flow diagram of while loop.

Syntax:

```
while expression:  
    statement(s)
```

Example:

```
1. cnt = 0  
2. while (cnt < 10):  
3.     print ("The count is:", cnt)  
4.     cnt = cnt + 1
```

Output:

```
The count is: 0  
The count is: 1  
The count is: 2  
The count is: 3  
The count is: 4  
The count is: 5  
The count is: 6  
The count is: 7  
The count is: 8  
The count is: 9
```

Nested Loop

A nested loop is a loop that is contained within another loop. We have the option of nesting a for loop inside a while loop, a while loop within of a for loop, a for loop inside of a for loop, or a while loop inside a while loop. To better grasp the nested loop, let us look at an example.

Syntax:

```
for var in sequence:  
    for var in sequence:  
        statements(s)  
        statements(s)
```

The following is an example of the Python programming language's syntax for the statement that creates a nested while loop:

while expression:

while expression:

statement(s)

statement(s)

Example 1:

```
1. # This program uses a nested for loop
2. i = 2
3. while(i < 20):
4.     j = 2
5.     while(j <= (i/j)):
6.         if not(i%j): break
7.         j = j + 1
8.     if (j > i/j) :
9.         print (i, " is prime")
10.    i = i + 1
```

Output:

```
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
13 is prime
17 is prime
19 is prime
```

Example 2:

```
1. #program to print a pattern
2. for i in range(1, 5):
3.     for j in range(i):
4.         print (j + 1, end = ' ')
5.     print ()
```

Output:

```
1
1 2
1 2 3
1 2 3 4
```

Example 3:

```
1. #program to print a pattern
2. for i in range (1, 5):
3.     for j in range(i):
4.         print (j + 1, end = ' ')
5.     print ()
```

Output:

```
1
1 2
1 2 3
1 2 3 4
```

Break Statement

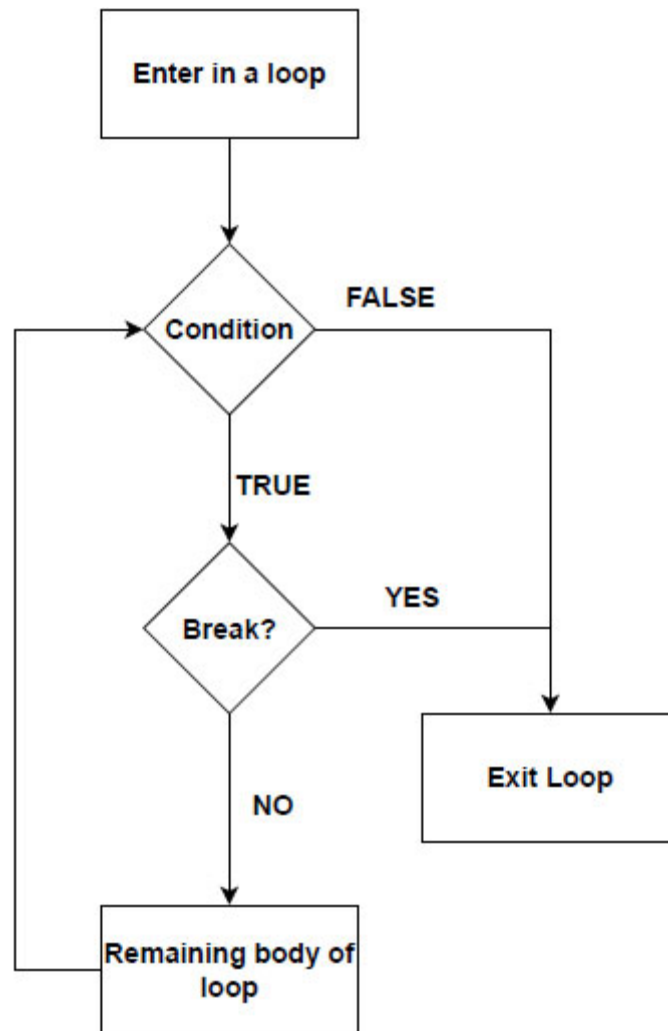
With the break statement, you can end a loop. When the break statement is run, the control will be moved out of the loop. Fig. (7) shows the flow chart for the break statement. If the break statement is inside a nested loop, it will end the loop that is closest to it.

Example:

```
1. numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9) #Declaring the tuple
2. num_sum = 0
3. count = 0
4. for x in numbers:
5.     num_sum = num_sum + x
6.     count = count + 1
7.     if count == 6:
8.         break
9. print ("Sum of first ",count,"integers is: ", num_sum)
```

Output:

```
Sum of first 6 integers is: 21
```

**Fig. (7))**

Flow diagram of break statement.

Continue Statement

It is utilized to bypass the remainder of the code included within a loop for the sole purpose of the current iteration. When a continue statement is present in a loop, the loop does not exit but continues with the subsequent iteration. Fig. (8) depicts the flow diagram of the continue statement in its many iterations.

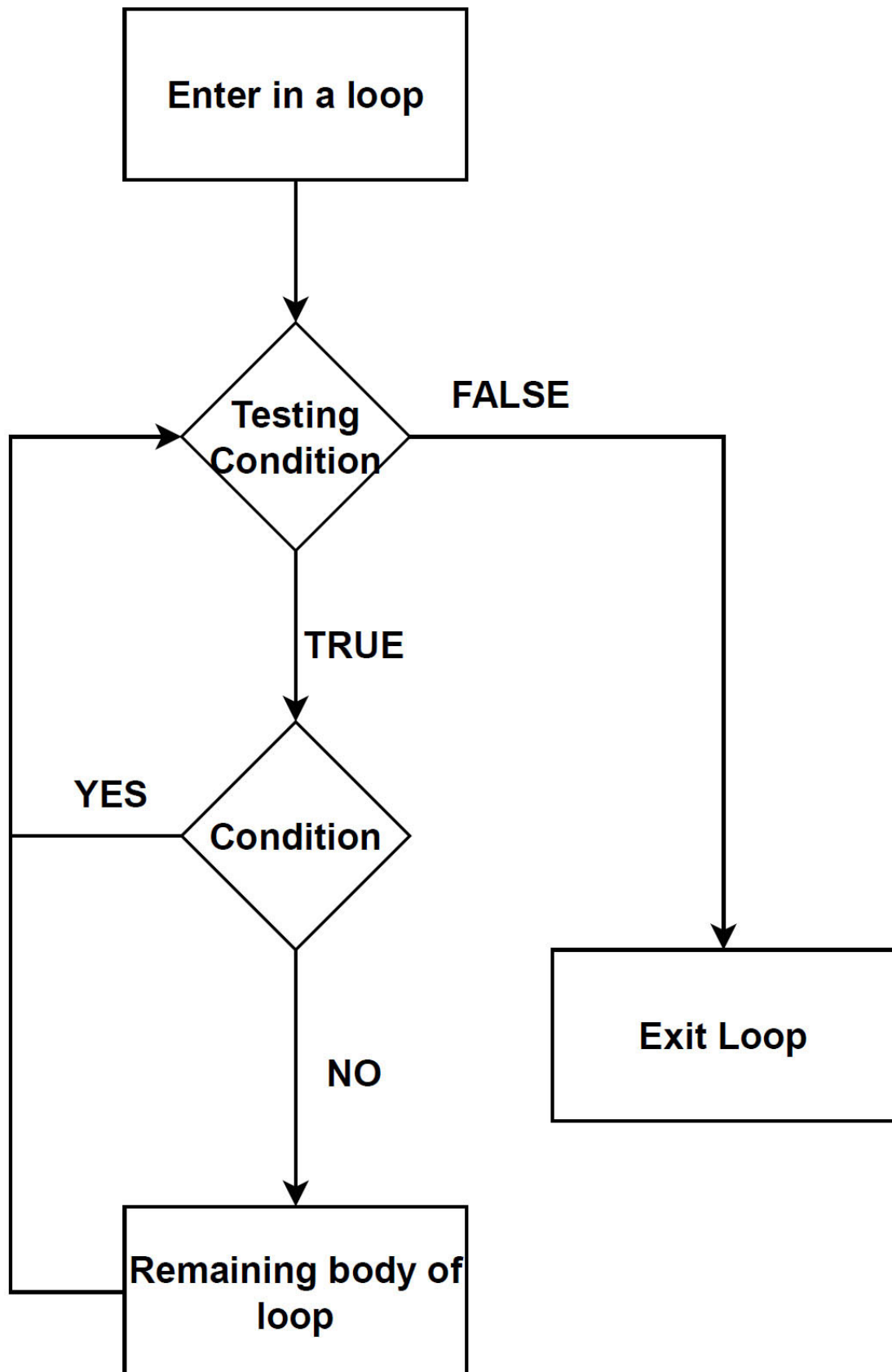


Fig. (8))

Flow diagram of continue statement.

Example 1:

```
1. for i in range(10):
2.     if i==5:
3.         continue
4.     print (i)
```

Output:

```
0
1
2
3
4
6
7
8
9
```

Example 2:

```
1. for x in range(7):
2.     if (x == 4 or x==6):
3.         continue
4.     print (x)
```

Output:

```
0
1
2
3
5
```

The Pass Statement

In Python, the term “null statement” refers to the *pass* statement. The most significant distinction between a comment and a pass statement is that the interpreter disregards the former, while the latter is not thus treated. In addition, when the pass is put into action, absolutely nothing takes place. It does not affect the operation. To better grasp the pass statement, let us look at an example.

Example 1:

```
1. a = 10
2. b = 20
3. if b > a:
4.     pass
```

Output:

No output

Example 2:

```
1. sequence = {'a', 'b', 'c', 'd', 'e', 'f'}
2. for x in sequence:
3.     pass
4.
```

Output:

No output

CONCLUDING REMARKS

This chapter covered several distinct categories of control statements, including sequential control structures, selection control structures, and repetition control structures. It provides an in-depth analysis of each control statement. We enunciate the functioning of the loops. As a result, this chapter features each control statement with an appropriate illustration and graphic. The next